AIDA roles vs Claude Code subagents (`/agents`)

AIDA positioning · best-effort comparison — live source: github.com/joemooney/aida

rendered 2026-07-10

Last updated: 2026-07-09 — Claude Code's `/agents` surface (the catalog of "Software Architect / Code Writer / Code Reviewer" presets, the tool-allowlist + model + prompt schema, the per-project `.claude/agents/` directory) is evolving. Re-verify against the current Claude Code docs before treating capability claims here as authoritative; if the surface drifts in a way that changes the layer story, update this doc.

The TL;DR: **Claude Code subagents are a within-conversation primitive — a specialized prompt + tool allowlist + isolated context window that ends with the chat. AIDA's roles are a cross-conversation workflow layer — full `claude` processes in their own git worktrees, holding leases, tied to specs in the requirement graph, with state that survives every conversation ending.** Different layers, not duplication. They compose: an AIDA role can spawn a subagent internally; AIDA is the layer above.

The worry that prompts this doc is a fair one. Claude Code's `/agents` ships presets named "Software Architect," "Code Writer," "Code Reviewer." AIDA ships roles named `planner`, `implementer`, `reviewer`. A reasonable reader sees that and asks: *am I getting the same thing twice?* The honest answer is **no** — but the rhyme is real, and saying so out loud is what makes the layer distinction stick.

What each one actually is

	Claude Code subagents (`/agents`)	AIDA roles + orchestrator
What it is	A specialized prompt + tool allowlist + model, with a fresh context window	A position in a lifecycle (`implementer` / `reviewer` / `advisor` / `planner`) held by a full `claude` process
Where it lives	Inside one Claude Code conversation; configured via `.claude/agents/*.md` or globally	A `claude` process in its own git worktree, with an `aida session` lease

	Claude Code subagents (`/agents`)	AIDA roles + orchestrator
Lifetime	Ephemeral — context window dies when the conversation ends; nothing persists	Cross-conversation — the lease, the queue entry, the spec status, and the trace comments survive every session ending
Anchoring	None — a subagent doesn't know about your requirements; it knows the prompt you gave it	Anchored to a SPEC-ID in the requirement graph (queue routing, lease tracking, status transitions, trace comments)
State	None outside the chat	Persisted in git (orphan `aida-store` branch, session manifests, lease files, trace comments in source)
Concurrency	Multiple subagents can run in parallel within one chat	Multiple roles run in parallel across separate worktrees, coordinated by leases + the queue
Discoverability	Visible only inside the originating conversation	Queryable via `aida queue list`, `aida session leases`, `aida role list`, MCP
Cost shape	Token spend inside the host conversation	Token spend across separate `claude` processes; coordination cost is the queue + lease infrastructure (one-time)
The shape	**A tool you delegate to**	**A workflow position with persistent state**

These are not the same kind of thing. A subagent is a *callable*; a role is a *position in a system*.

The separating test

The clearest way to feel the layer distinction is to imagine the same nominal job done two ways:

> A Claude Code "Code Reviewer" subagent reviews code you hand it, in the moment; the result lives only in that conversation.
>
> An AIDA reviewer role reviews a specific PR tied to a specific spec, writes a verdict file that drives an orchestrator merge phase, flips the spec's status in the graph, fires the auto-bump — durable, queryable, surviving every conversation ending.

Both produce review output. Only one of them produces output that *another* session, *another* day, can find. That difference is the layer.

Why AIDA's roles can't just BE subagents

A natural follow-up: *"if subagents are a built-in primitive, why doesn't AIDA just use them for its roles?"* The answer is that the things AIDA's roles are made of don't exist at the subagent layer:

- **No persistence.** A subagent's context window is gone the moment the host conversation ends. You cannot build a queue out of a thing whose memory dies.
- **No graph anchoring.** A subagent doesn't know what a SPEC-ID is. Routing work to "the reviewer for STORY-86" requires a primitive that ties an actor to a graph node — subagents have no such tie.
- **No cross-session lifecycle.** AIDA's 6-phase orchestrator (commit → PR → CI → review → merge → bump) spans hours-to-days and crosses every session boundary in between. Subagents live and die inside one chat; they cannot be the substrate of a multi-day workflow.
- **No lease coordination.** Two implementers picking up the same spec from the queue is a race that the lease system prevents. Subagents share their host conversation's context — there is no "between subagents" layer to put a lease in.
- **No durable artifacts.** A reviewer subagent's verdict is a chat message. An AIDA reviewer's verdict is a file on disk that drives an orchestrator merge phase — and a structural self-merge guard blocks the implementer from merging its own work past that reviewer. Different substrates, different leverage.

AIDA's defensible niche **is** the persistent graph + the cross-session lifecycle. You cannot build it out of stateless task-runners — wrong layer. The queue, the leases, the trace comments, the auto-bump, the MCP exposure — none of these are things a within-conversation primitive can carry.

They compose

The complementary picture: **an AIDA role can spawn a subagent internally.** A `claude` process held by the implementer role is itself a Claude Code session — it can invoke `/agents` like any other session. Concrete patterns:

- An **implementer** running a "Code Reviewer" subagent against the PR diff as a pre-`/aida-pr` self-check, before the AIDA reviewer role ever sees it.
- A **planner** delegating "scan this directory for affected callsites" to a subagent with a tight tool allowlist, keeping the host context clean.
- An **advisor** spawning a "Software Architect" subagent for a quick brainstorm without polluting the long-running advisory conversation.

In every case, the subagent is **a tool the role uses**, not a replacement for the role. Subagents make AIDA roles *better* — sharper focus, cleaner context windows, lower token spend per turn. AIDA is the layer above them.

The right mental model is a two-level stack: **subagents** are the within-session specialization layer; **roles** are the cross-session workflow layer. Both are useful. Neither replaces the other.

The name-rhyme caveat

It needs saying out loud: **"Software Architect / Code Writer / Code Reviewer" ≈ AIDA planner / implementer / reviewer is surface rhyme, not substance.** The naming overlap is real, and it is the single biggest source of the "am I reinventing the wheel?" worry that prompts this doc.

What the rhyme captures (genuinely): both systems carve up code work into roughly similar functional buckets. Planning is a different mental mode from writing, which is different from reviewing. Both systems noticed this and named the buckets.

What the rhyme hides (substantively): the **scope** of each bucket. A Claude Code "Code Reviewer" reviews whatever code you hand it in a chat. An AIDA reviewer reviews a specific PR for a specific spec, writes a verdict file, drives a merge phase, flips a status. Same word, different layer. The presence of three buckets in each system tells you nothing about whether they live at the same layer; the *lifecycle* of each bucket tells you everything.

If you only read the names, the answer to *"am I reinventing?"* looks like *"yes."* If you read what each name produces and where it lives, the answer is *"no — different layers."*

AIDA's own discipline: don't reinvent the within-session parts

A positioning doc is also a watch-item for the project it positions. The corollary of "AIDA is the cross-session layer" is **"AIDA must not reinvent the within-session layer that `/agents` already serves."**

The concrete rule: if AIDA finds itself building "spawn a quick specialized in-context helper to do X within the current session," **stop — that's `/agents`.** AIDA's niche is strictly the cross-session graph + lifecycle layer; below that line, defer to Claude Code primitives.

Worked-out cases of the discipline:

- **AIDA does NOT need** its own in-session specialization mechanism (a "spawn a focused sub-thread for this code-scan" feature). `/agents` already does it. AIDA shells out to it from inside a role's `claude` process.
- **AIDA does NOT need** its own tool-allowlist scoping for in-conversation delegation. `/agents` already does it.
- **AIDA does NOT need** its own context-isolation primitive. `/agents` already does it.
- **AIDA DOES need** its own queue, lease, worktree, orchestrator, trace-graph, MCP layer — none of which `/agents` provides or attempts to.

The test, when adding any feature: *"does this give the session-level work a primitive Claude Code already provides, or does it give the cross-session workflow a primitive nothing else does?"* The first kind is yak shaving; the second kind is AIDA's job.

Honest scope statement

AIDA's value proposition vs Claude Code subagents is **not** *"better roles than `/agents`."* The two systems live at different layers of the stack; comparing them as substitutes is the wrong frame. The defensible thing AIDA brings is *"a workflow layer above subagents that has graph anchoring, persistent state, queue routing, and lifecycle bookkeeping — none of which a within-conversation primitive can carry."* That is a complementary capability, not a replacement.

If a Claude Code user has not yet hit the cross-session pain — *"yesterday's session decided X and today's session doesn't know," "two agents touched the same code and silently disagreed," "the PR merged but the requirement is still in 'in-progress' because nobody flipped it"* — subagents alone may genuinely be all they need. The threshold for adding AIDA is not the number of subagents in the project; it is the first time a piece of work *crosses a session boundary and needs the next session to know about it.*

When that threshold lands, AIDA composes underneath: subagents stay where they are, working inside whichever role-held `claude` process needs them.

See also

- vs-ultrareview.md — same shape of comparison applied to Claude Code's cloud review surface.
- vs-ultraplan.md — same shape of comparison applied to Claude Code's cloud planning surface.
- vs-karpathy-md.md — the "is structured markdown enough?" question at the other end of the spectrum.
- `docs/aida/discipline/advisor-role.md` — the advisor role consults this directory when a user asks a "where does AIDA fit / vs X" question, rather than improvising.
- `OVERVIEW.md` "Public face: the TUI is the product" — the broader framing for why AIDA's value is below the visible surface.